Algoritmo Aho-Corasick

Autor: Facundo Nicolás García Zárate
Materia: Taller de Tecnologías de Producción de Software.
Opción C - Técnicas y Estrategias para la resolución de problemas
Profesora: Alejandra Schiavoni

Índice:

Resumen..... 3
1. Introducción..... 4
1.1 Motivación..... 4
2. Conceptos previos..... 4
2.1 Terminología y notación..... 4
2.2 Problema con el enfoque ingenuo..... 5
2.3 Tries..... 5
3. El algoritmo Aho-Corasick..... 5
3.1 Construcción del trie de patrones..... 6
3.2 Enlaces de sufijo..... 6
3.2.1 Algoritmo de construcción de enlaces de sufijo..... 6
3.3 Enlaces de salida..... 7
3.5 El algoritmo de búsqueda..... 7
4. Análisis de complejidad..... 7
4.1 Fase de preprocesamiento..... 7
4.2 Fase de búsqueda..... 7
4.3 Comparación con las alternativas..... 8
5. Implementación en Python..... 8
6. Aplicaciones prácticas..... 10
7. Conclusiones..... 10
8. Bibliografía..... 11

Resumen

En muchas aplicaciones de recuperación de información y edición de texto es necesario poder localizar de una manera rápida algunas o todas las ocurrencias de patrones de palabras y frases especificadas por el usuario en un texto. Aho-Corasick es un algoritmo simple y eficiente que localiza todas las ocurrencias de cualquiera de un número finito de palabras clave y frases en una cadena de texto arbitraria. Fue desarrollado por Alfred V. Aho y Margaret J. Corasick en 1975 y publicado en el artículo «Efficient String Matching: An Aid to Bibliographic Search».

La principal ventaja del algoritmo radica en su complejidad temporal: dado un conjunto de patrones de longitud total n y un texto de longitud m, la fase de preprocesamiento toma tiempo O(n) y la fase de búsqueda toma tiempo O(m + z), donde z es el número de ocurrencias encontradas. Esto lo hace óptimo para escenarios donde se dispone de un conjunto fijo de patrones y se deben procesar múltiples textos.

En esta monografía se presentan los fundamentos teóricos del algoritmo, sus tres componentes principales (el trie de patrones, los enlaces de sufijo y los enlaces de salida), el análisis de complejidad, su implementación y aplicaciones prácticas.

1. Introducción.

El problema de búsqueda de cadenas, en su forma más básica, consiste en encontrar todas las ocurrencias de uno o más patrones dentro de un texto dado. Si bien existen soluciones simples mediante fuerza bruta, estas son ineficientes e inviables cuando el número de patrones o la longitud del texto son grandes.
Consideremos el siguiente problema: dado un texto T y k cadenas no vacías $P_1, P_2, \ldots, P_k$ (los patrones), encontrar todas las ocurrencias de cada P_i en T.
Este es un escenario común en aplicaciones reales (más adelante voy a profundizar en estos casos).
El enfoque ingenuo, que verifica cada patrón en cada posición del texto, tiene complejidad O(mn), donde m es la longitud del texto y n es la suma de las longitudes de los patrones.
El algoritmo Aho-Corasick resuelve este problema construyendo un autómata finito determinista a partir del conjunto de patrones, lo que permite procesar el texto en una sola pasada con tiempo lineal O(m + z).

1.1 Motivación

Alfred V. Aho y Margaret J. Corasick publicaron su algoritmo en 1975 en el contexto de la construcción de sistemas de búsqueda bibliográfica. En el paper original ellos mencionan que el algoritmo fue desarrollado para acelerar la búsqueda simultánea de palabras clave en grandes colecciones de texto, un problema común en los sistemas de recuperación de información de la época.

2. Conceptos previos

2.1 Terminología y notación

Para el análisis del algoritmo se utiliza la siguiente terminología :

- $m = |T|$: longitud del texto en el que se buscarán los patrones.
- $n = |P_1| + |P_2| + \ldots + |P_k|$: longitud total de todos los patrones.
- L_{max}: longitud del patrón más largo.
- Σ: alfabeto de los caracteres. Se asume $|\Sigma|$ constante.
- z: número total de ocurrencias encontradas.

2.2 Problema con el enfoque ingenuo

El algoritmo ingenuo recorre cada posición del texto T y, para cada posición, verifica uno por uno si algún patrón P_i se encuentra en esa posición. En el peor caso, esto implica examinar *n* caracteres por cada una de las *m* posiciones del texto, resultando en una complejidad de O(mn).
De esta manera se vería el pseudocódigo de la solución ingenua:

```
por cada posición en T:
    por cada pattern string Pi:
        verificar si Pi aparece en esa posición
```

Un ejemplo que evidencia este comportamiento del peor caso es buscar los patrones {a, aa, aaa, aaaa} en el texto aaaaa…a. En cada posición se verifican todos los patrones, y cada comparación puede extenderse hasta la longitud del patrón más largo.

2.3 Tries

El algoritmo Aho-Corasick parte de una estructura de datos llamada trie (del inglés re**trie**val). Un trie es un árbol donde cada nodo representa un prefijo de alguno de los patrones almacenados.
Un trie construido sobre los patrones $P_1, \ldots, P_k$ presenta las siguientes propiedades:

- La raíz representa la cadena vacía ε.
- Cada nodo a profundidad *d* representa un prefijo de longitud *d*.
- Cada patrón P_i corresponde a exactamente un nodo terminal (en el contexto de trie, un nodo terminal no se refiere a una hoja del árbol, sino que a aquel nodo el cual marca el final de un patrón).
- El trie tiene *n* nodos (en el peor caso) y puede construirse en tiempo O(n).

Usando el trie para la búsqueda, la complejidad mejora a $O(m.L_{max})$, lo que es una mejora significativa cuando $L_{max} < n$. Sin embargo, aún no es óptima.

3. El algoritmo Aho-Corasick

El algoritmo Aho-Corasick extiende el trie base con dos tipos adicionales de enlace que permiten la búsqueda en tiempo O(m + z). La idea clave es que,

cuando la búsqueda falla en algún nodo del trie (es decir, no hay transición válida para el carácter actual del texto), en lugar de volver al inicio, el autómata puede saltar a un estado que preserve la mayor cantidad de contexto posible.

3.1 Construcción del trie de patrones

El primer paso del algoritmo es insertar todos los patrones $P_1, ..., P_k$ en un trie. Para cada patrón P_i, se sigue el camino desde la raíz creando nodos nuevos cuando sea necesario. Los nodos terminales (que corresponden al final de un patrón) se marcan con la información del patrón que representan.

3.2 Enlaces de sufijo

El componente más importante del algoritmo es el enlace de sufijo (también llamado enlace de falla o failure link). Si el nodo *v* representa la cadena α, su enlace de sufijo apunta al nodo que representa max{ω : ω es sufijo de α y ω está en el trie}. Si no hay ningún sufijo de α en el trie (distinto de ε), el enlace apunta a la raíz.
La idea de extender el trie agregando los enlaces de sufijo es muy inteligente, pensándolo intuitivamente podemos llegar a la siguiente conclusión: cuando se está en un nodo *v* intentando leer el carácter *c* del texto y el nodo *v* no tiene ningún hijo etiquetado con *c*, en lugar de volver a la raíz y perder todo el contexto, se sigue el enlace de sufijo y se intenta nuevamente. Esto preserva el sufijo más largo del texto ya procesado que podría ser un prefijo de algún otro patrón en el trie.

3.2.1 Algoritmo de construcción de enlaces de sufijo

Los enlaces de sufijo se construyen mediante una búsqueda en anchura (BFS) sobre el trie. Para calcular el enlace de sufijo de un nodo v, se arranca desde el failure link de su padre y se siguen failure links hasta encontrar un nodo que tenga un hijo con el carácter correcto, o hasta llegar a la raíz. BFS garantiza que cuando se procesa un nodo, su padre ya fue procesado y su failure link ya está disponible. La complejidad de este paso es O(n).

3.3 Enlaces de salida

Consideremos los patrones {"he", "she"} y el texto "she". Cuando se procesa el último carácter, el autómata llega al nodo "she", que es terminal, por lo que se reporta el patrón "she". Pero "he" también ocurre en el texto y no sería reportado, porque el nodo "she" no es terminal para "he". Sin embargo, el enlace de sufijo de "she" apunta al nodo "he", que sí es terminal.
El enlace de salida (también llamado output link) resuelve esto: es un puntero directo al primer nodo terminal que se encuentra siguiendo la cadena de enlaces de sufijo desde el nodo actual. Con este enlace, al llegar a cualquier nodo durante la búsqueda, sólo es necesario seguirlo para encontrar el siguiente patrón a reportar, sin recorrer toda la cadena de sufijos en cada paso.

3.5 El algoritmo de búsqueda

En cada posición del texto se lee un carácter y se intenta avanzar en el trie. Si el nodo actual tiene una transición válida, se avanza directamente. Si no, se sigue el enlace de sufijo y se intenta de nuevo, repitiendo hasta encontrar una transición válida o llegar a la raíz. Una vez que se avanza al nuevo nodo, se reportan los patrones que terminan ahí, y además se sigue el enlace de salida para reportar cualquier otro patrón que esté escondido en la cadena de sufijos.

4. Análisis de complejidad

4.1 Fase de preprocesamiento

El preprocesamiento del algoritmo de Aho-Corasick consta de tres etapas, cada una con su complejidad:

- Construcción del trie: tiempo O(n).
- Construcción de enlaces de sufijo: tiempo O(n)
- Construcción de enlaces de salida: tiempo O(n).

4.2 Fase de búsqueda

La búsqueda recorre los *m* caracteres del texto manteniendo un estado actual en el autómata. Para cada carácter hay dos casos: si el nodo actual tiene una transición válida, se avanza directamente; si no, se siguen enlaces de sufijo

hasta encontrar un nodo que sí la tenga, o hasta llegar a la raíz. Cada vez que se sigue un enlace de sufijo se sube en profundidad en el trie, y eso solo puede ocurrir tantas veces como se bajó avanzando por el texto. Por tanto, la complejidad de búsqueda es O(m + z), siendo *z* el costo que conlleva el reporte de ocurrencias.

4.3 Comparación con las alternativas

	Fuerza bruta (solución ingenua)	Búsqueda con trie simple	Aho-Corasick
Preprocesamiento	O(1)	O(n)	O(n)
Búsqueda	O(mn)	$O(m.L_{max})$	O(m + z)

5. Implementación en Python

```
from collections import deque

class Node:
    def __init__(self):
        self.outputs = set()
        self.children = {}
        self.failure_link = None

    def has_child(self, key):
        return key in self.children

    def get_child(self, key):
        return self.children.get(key)

    def set_child(self, key, node):
        self.children[key] = node

    def add_output(self, output):
        self.outputs.add(output)

    def copy_outputs(self, node):
        for o in node.outputs:
            self.outputs.add(o)
```

```python
class AhoCorasick:
    def __init__(self, patterns):
        # Creamos el trie
        self.root = Node()

        for pattern in patterns:
            curr_node = self.root
            for key in pattern:
                if not curr_node.has_child(key):
                    curr_node.set_child(key, Node())
                curr_node = curr_node.get_child(key)
            curr_node.add_output(pattern)
        # Inicializamos los enlaces de sufijo para la raíz y sus hijos
        self.root.failure_link = self.root
        queue = deque()

        for child in self.root.children.values():
            child.failure_link = self.root
            queue.append(child)

        # Inicializamos los enlaces de sufijo y de salida para los demás
nodos.
        while queue:
            curr_node = queue.popleft()

            for key, child in curr_node.children.items():
                queue.append(child)

                n = curr_node.failure_link
                while not n.has_child(key) and n != self.root:
                    n = n.failure_link

                if n.has_child(key):
                    child.failure_link = n.get_child(key)
                else:
                    child.failure_link = self.root

                child.copy_outputs(child.failure_link)

    def search(self, text):
        found = []
        state = self.root
        i = 0

        while i < len(text):
```

```
            c = text[i]

            if state.has_child(c):
                state = state.get_child(c)
                i += 1

                if state.outputs:
                    for val in state.outputs:
                        found.append({
                            "pos": i - len(val),
                            "val": val
                        })

            elif state == self.root:
                i += 1
            else:
                state = state.failure_link

        return found
```

6. Aplicaciones prácticas

- Seguridad informática: es esencial para detectar patrones maliciosos en el tráfico y los logs en tiempo real.
- Bioinformática: tiene un rol crucial a la hora de buscar rápidamente secuencias de ADN en bases de datos genómicas.
- Filtrado de contenido: detección de palabras prohibidas en plataformas de mensajería y redes sociales, donde hay un diccionario fijo de términos a detectar y un flujo constante de mensajes nuevos a analizar.
- Motores de búsqueda de texto: Herramientas como grep en sus variantes modernas usan algoritmos derivados para búsqueda eficiente de múltiples patrones.

7. Conclusiones

El algoritmo de Aho-Corasick resuelve el problema de búsqueda simultánea de múltiples patrones en texto de manera óptima. Su complejidad ⟨O(n), O(m + z)⟩ lo hace especialmente eficiente cuando se dispone de un conjunto fijo de patrones sobre el que se realizan múltiples búsquedas, ya que el costo de construcción del autómata se paga una sola vez.

El algoritmo se distingue por cómo combina tres ideas: el trie como estructura base para representar los patrones, los enlaces de sufijo para evitar trabajo redundante cuando una comparación falla, y los enlaces de salida para garantizar que ninguna ocurrencia quede sin reportar.
Desde su publicación en 1975 el algoritmo sigue siendo relevante en aplicaciones de alto impacto como la detección de intrusiones en redes y la búsqueda de secuencias en genomas, áreas donde la escala del problema hace que la diferencia entre una solución lineal y una cuadrática sea la diferencia entre un sistema funcional y uno inviable.

8. Bibliografía

Aho, A. V. y Corasick, M. J. (1975). Efficient string matching: An aid to bibliographic search. Communications of the ACM, 18(6), 333–340.
Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. (2009). Introduction to Algorithms (3ra ed., Cap. 32). MIT Press.
Sedgewick, R. y Wayne, K. (2011). Algorithms (4ta ed., Cap. 5). Addison-Wesley.